OLYMPIX

SECURITY VERIFICATION REPORT

Fables

BugScout

A negative-assurance, symbolic-execution-driven BugScout engagement against **FablesFi/fables** — Olympix enumerated execution paths across the 2 highest-value contracts, generated concrete adversarial Foundry tests under 3 attack strategies, and ran every one — with a runnable artifact behind every number.

PATHS EXPLORED	FEASIBLE	TEST CASES	FILES	EXPLOITS
3,375	**1,329**	**985**	**8**	**0**

Contents

01 Executive summary
02 Why this matters
03 How the verification engine works
04 BugScout results
05 Path breakdown
06 Paths by call ordering
07 System under test
08 Adversarial strategies in depth
09 Run profile

01 Executive summary

Olympix ran its symbolic-execution-driven verification engine against the Fables protocol, probing the core state-changing surface with concrete adversarial execution.

3,375
execution paths enumerated across the 2 highest-value contracts

0
successful exploits across 985 adversarial test cases & 3 attack strategies

985
adversarial test cases generated & executed

8
runnable Foundry test files produced

Symbolic-execution-driven verification. Explored 3,375 execution paths across the 2 highest-value contracts, kept the 1,329 feasible (satisfiable) ones, generated and ran 985 targeted test cases under 3 adversarial strategies, and produced **zero successful exploits** — a strong negative-assurance result for the core surface.

BugScout

3,375 paths · 1,329 feasible · 985 test cases · 8 generated Foundry files

0 exploits

across all 3 attack strategies

Engagement at a glance

Target	FablesFi/fables
Commit	`85959477171f8f2f7030d40a6996ed138ea0dce6`
Branch	`olympix`
Scope	2 contract(s) verified in depth · 3,375 paths enumerated
Toolchain	Olympix BugScout (Foundry + symbolic execution)
Run	session `79939a21-05b3-4a3f-9d25-c3ffde466cda` · 49 minutes wall-clock · November 11, 2026 UTC

Maturity note — the verification engine is a work in progress

The verification engine is under active development. The 3 adversarial strategies exercised in this run are the engine's current set; additional strategies and deeper multi-call sequences land with each release, so today's result is a floor on coverage, not a ceiling.

02 Why this matters

Negative assurance you can show your community

“We verified it” is easy to claim. 3,375 symbolically-enumerated paths, 985 concrete adversarial test cases and 0 successful exploits is a claim with artifacts behind every number — every test case is a runnable Foundry file in this package.

Real exploit attempts, not warnings

Each generated test is a genuine attack — funding the contract, pranking an attacker, deploying real reentrancy harnesses — whose assertions fail if the attack succeeds. A clean run means the attacks were tried and held.

Path-aware, not blind

Symbolic execution enumerates reachable paths and an SMT solver keeps only the feasible ones, so the adversarial tests concentrate on inputs that can actually occur rather than random noise.

Multi-call, not single-shot

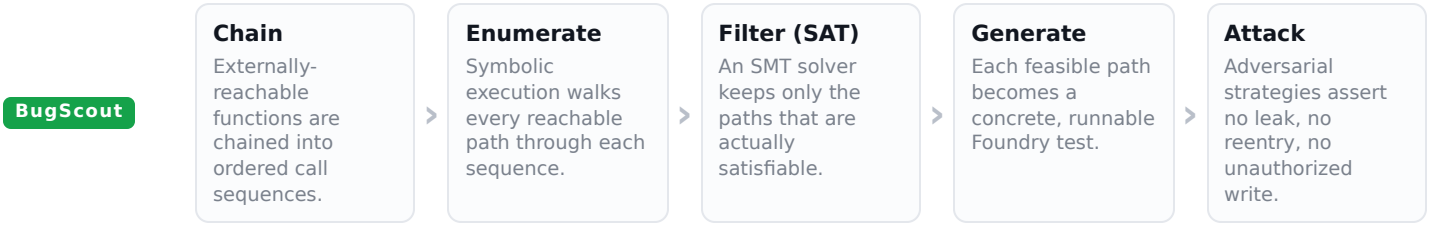
The engine chains externally-reachable functions into ordered call sequences and carries state from one call to the next, so bugs that only appear after a specific ordering of transactions are in scope.

Fast enough for every release

Specification to result took 49 minutes. This can run on every release candidate, not once per audit cycle.

03 How the verification engine works

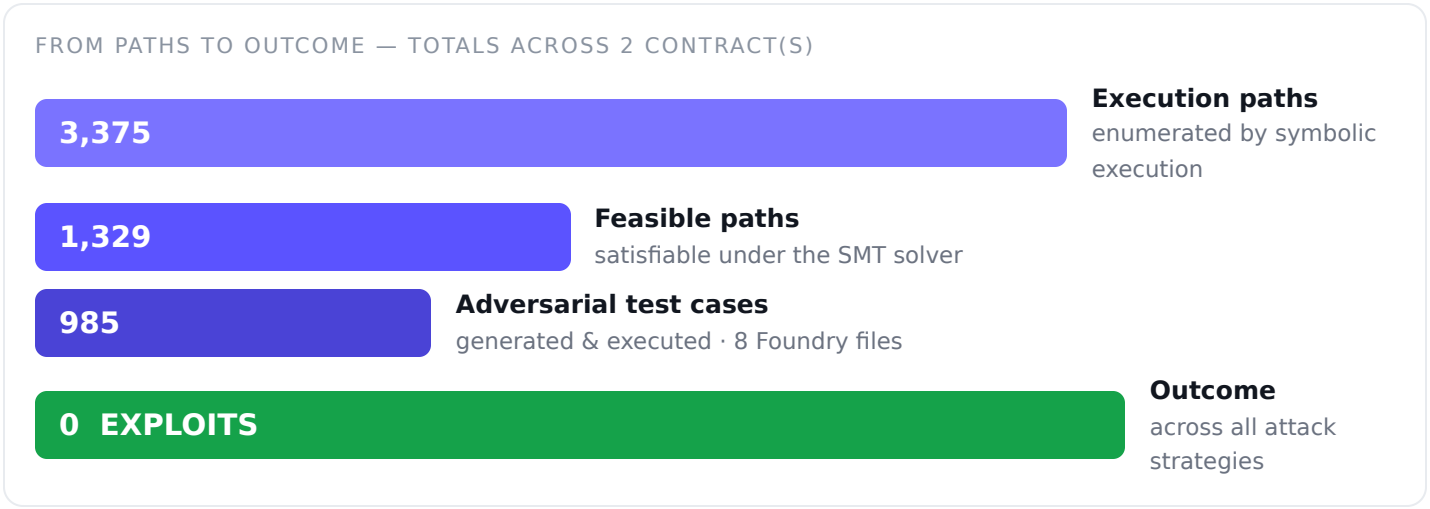
The engine starts from whole-contract symbolic execution, turns feasible paths into concrete adversarial tests, and runs every one.



BugScout executes thousands of concrete adversarial transactions and shows what attackers actually achieve against the contract under realistic conditions.

04 BugScout results

For each contract, Olympix enumerated execution paths, kept the feasible ones, and generated concrete Foundry tests that drive every feasible path under each adversarial strategy. Each test is a real exploit attempt whose assertions fail if the attack succeeds.



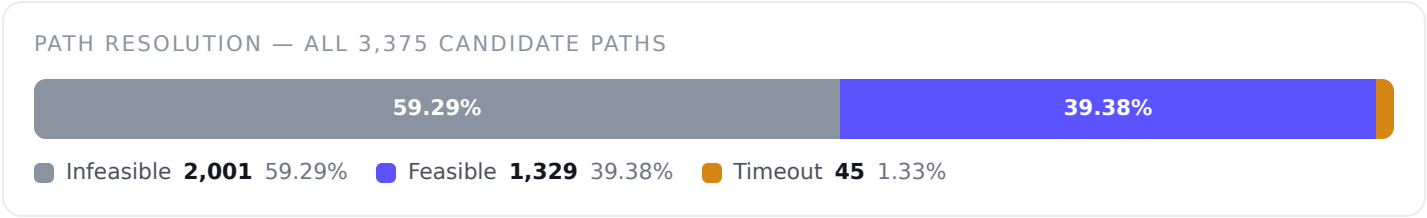
Results by contract

Contract	Paths	Feasible	Test cases	Files	Exploits
FablesStableHookLite	1,662	663	494	4	0
FablesStableHookLiteTyped	1,713	666	491	4	0
Total	**3,375**	**1,329**	**985**	**8**	**0**



05 Path breakdown

Symbolic execution enumerated 3,375 candidate execution paths. This section explains what happened to each one — the funnel from candidate path to runnable adversarial test.



Why most paths are infeasible — and why that is a positive result

An infeasible path is one the solver *proved* unreachable: asked for an input assignment in which an attacker-controlled caller reaches the function body, it showed no such assignment exists. A high infeasible share against attacker preconditions is direct evidence that the guards on privileged entry points hold at the constraint level — it is not a gap in coverage.

Resolution	Paths	Share	What it means
Infeasible	2,001	59.29%	Path constraint proven unsatisfiable — unreachable under the modelled preconditions.
Feasible	1,329	39.38%	Solver produced a concrete input assignment; the path became a runnable test.
Timeout	45	1.33%	Solver exceeded its budget for this path; reachability undetermined.
Total	**3,375**	**100%**	

Of the enumerated paths, 1,329 carried forward into concrete test generation; the remaining 2,046 were not reachable under the modelled attacker preconditions.

06 Paths by call ordering

The engine does not test functions in isolation. It builds **call sequences** — ordered chains of externally-reachable functions — and explores every path through the sequence with state carried forward from one call to the next. This is where multi-transaction bugs live.

65
distinct call orderings explored

2
maximum sequence depth (calls per chain)

51
mean paths per ordering

1,329
feasible paths carried into test generation

Every ordering, with its own path funnel

#	Contract	Call ordering	Paths	Feasible	Yield
1	FablesStableHookLite	`constructor` → `.fallback`	258	29	11.24%
2	FablesStableHookLiteTyped	`constructor` → `.fallback`	251	29	11.55%
3	FablesStableHookLite	`constructor` → `setClaimFee`	248	108	43.55%
4	FablesStableHookLiteTyped	`constructor` → `setClaimFee`	248	108	43.55%
5	FablesStableHookLite	`constructor` → `setDistributor`	208	116	55.77%
6	FablesStableHookLiteTyped	`constructor` → `setDistributor`	208	116	55.77%
7	FablesStableHookLite	`constructor` → `setFFFeeSink`	112	52	46.43%
8	FablesStableHookLiteTyped	`constructor` → `setFFFeeSink`	112	52	46.43%
9	FablesStableHookLite	`constructor` → `setClaimFeeBps`	100	8	8.00%
10	FablesStableHookLiteTyped	`constructor` → `setClaimFeeBps`	100	8	8.00%
11	FablesStableHookLite	`constructor` → `configureStablePool`	72	34	47.22%
12	FablesStableHookLiteTyped	`constructor` → `configureStablePool`	72	34	47.22%
13	FablesStableHookLite	`constructor` → `setBallotGate`	68	36	52.94%
14	FablesStableHookLiteTyped	`constructor` → `setBallotGate`	68	36	52.94%
15	FablesStableHookLite	`constructor` → `setDiscount`	60	32	53.33%
50 further ordering(s)			1,190	531	44.62%
Total			**3,375**	**1,329**	**39.38%**

Showing the 15 orderings with the highest path volume; the remaining 50 are aggregated into the row above so the totals stay complete.

WHERE PATHS FLOW, BY POSITION IN THE CALL SEQUENCE

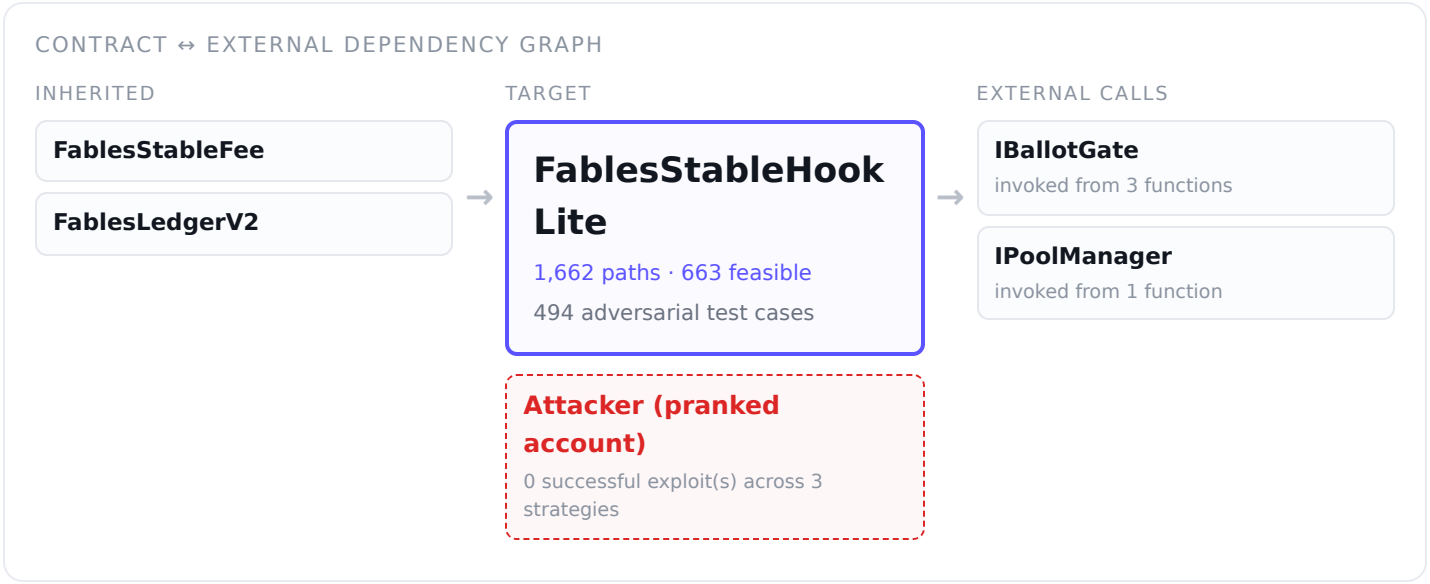
1ST CALL	2ND CALL
constructor 3,375 paths · 1,329 feasible	**.fallback** 509 paths · 58 feasible
	setClaimFee 496 paths · 216 feasible
	setDistributor 416 paths · 232 feasible
	setFFFeeSink 224 paths · 104 feasible
	setClaimFeeBps 200 paths · 16 feasible
	configureStablePool 144 paths · 68 feasible
	setBallotGate 136 paths · 72 feasible
	setDiscount 120 paths · 64 feasible

A function appearing at more than one position is counted at each position it occupies; column totals therefore exceed the run total.

07 System under test

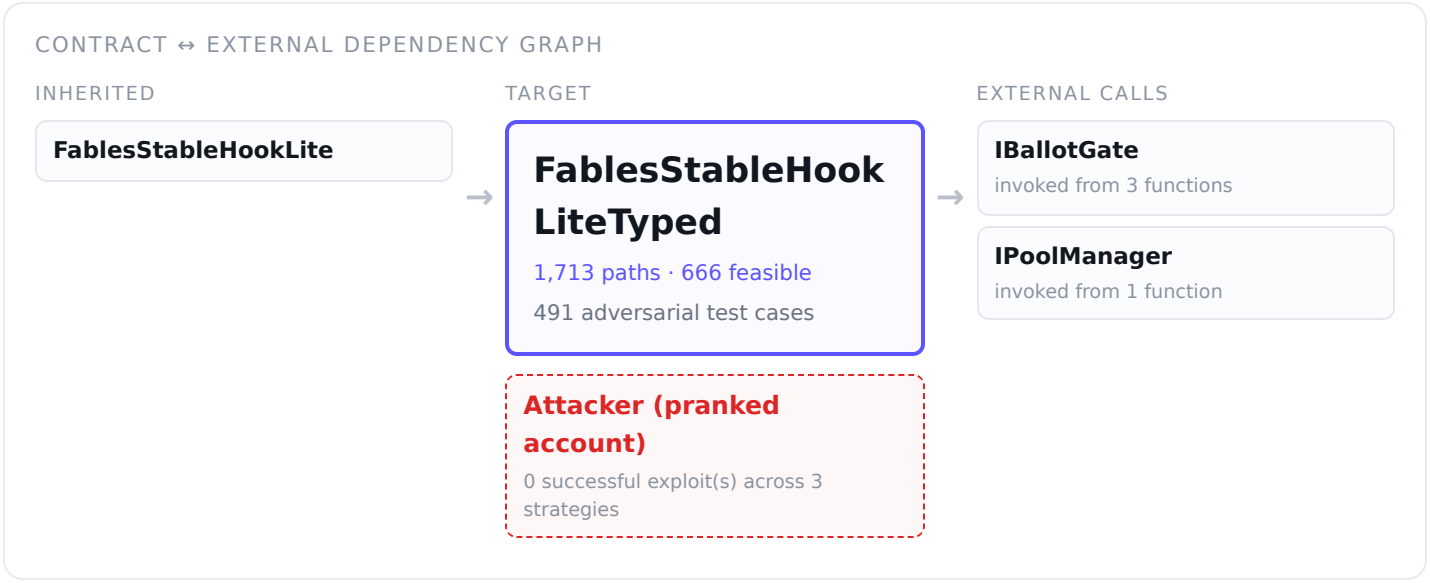
What each verified contract is wired to, and where the trust boundaries sit. Every external contract the target invokes is an assumption the generated harnesses had to model.

FablesStableHookLite



Component	Relationship	Invoked from	Modelled as
`IBallotGate`	External Call	3	External interface — calls to it are an assumption the harness models.
`IPoolManager`	External Call	1	External interface — calls to it are an assumption the harness models.
`FablesStableFee`	Inheritance	—	Inherited surface — included in path exploration as part of the target.
`FablesLedgerV2`	Inheritance	—	Inherited surface — included in path exploration as part of the target.

FablesStableHookLiteTyped



Component	Relationship	Invoked from	Modelled as
`IBallotGate`	External Call	3	External interface — calls to it are an assumption the harness models.
`IPoolManager`	External Call	1	External interface — calls to it are an assumption the harness models.
`FablesStableHookLite`	Inheritance	—	Inherited surface — included in path exploration as part of the target.

08 Adversarial strategies in depth

Each strategy is a distinct threat model with its own harness, preconditions and failing assertion. Below: what it assumes, what it builds, what it asserts, and how to read the result.

1 Reentrancy

Reentrancy

0 / 140 exploited

THREAT MODEL	An external call hands control to attacker code before the caller's state is settled, letting the attacker re-enter and exploit a half-updated contract.
HARNESS	Deploys a genuine attacker contract whose `receive()` calls back into the target and emits a detection event — a real reentrant control-flow handoff, not a flat call sequence.
ASSERTION	The test scans emitted logs for the detection event. If the callback ever re-entered a state-changing function, the event fired and the test fails.
READING IT	**0 exploited** across 140 concrete attempts: every re-entry attempt was rejected before any partially-updated state could be observed.

TEST CASES	FOUNDRY FILES	CONTRACTS COVERED	EXPLOITS
140	**4**	**2**	**0**

2 State-modification authority

UnauthorizedStateModification

0 / 845 observed

WHAT IT MAPS	Which callers can mutate which protocol state. Read this as a map of the contract's effective write authority, not as a list of defects — a privileged caller writing the state it governs is the system working as designed.
HARNESS	Deploys the contract from a concrete constructor solution, then replays each feasible path both as an unrelated account holding no role and as the caller the solver had to select in order to satisfy that path's guards.
WHAT IT RECORDS	Storage is recorded across the call with `vm.startStateDiffRecording()`, and every slot whose value actually changed is attributed to the caller that changed it. Comparing values rather than counting writes means a write that stores a value back onto itself is correctly read as no change.
READING IT	**0 observations** across 845 concrete attempts: no caller reached state beyond the authority its guards grant it.

TEST CASES	FOUNDRY FILES	CONTRACTS COVERED	OBSERVATIONS
845	**4**	**2**	**0**

3 Unchecked external call

`UncheckedExternalCall`

0 / 0 exploited

THREAT MODEL	An external call's return value is ignored or mishandled, so a silent failure is treated as success and corrupts downstream accounting.
HARNESS	Drives paths where external-call results feed back into state, substituting failing and hostile callees for the external target.
ASSERTION	State written after the external call must be consistent with the call's actual outcome; a failed call must not leave the contract believing it succeeded.
READING IT	**0 exploited** across 0 concrete attempts: no path let an unchecked external result corrupt accounting.

TEST CASES	FOUNDRY FILES	CONTRACTS COVERED	EXPLOITS
0	0	2	0

09 Run profile

How the run progressed over 49 minutes, and the parameters it ran under.

- **Started** +0s
- **Running** +1m 46s
  Running fuzzing on contract FablesStableHookLite
- **Running** +21m 29s
  Running fuzzing on contract FablesStableHookLiteTyped
- **Finished** +48m 57s

Contracts verified	2
Requested chain length	1 calls per chain
Call orderings explored	65
Maximum sequence depth	2 calls per chain
Wall-clock	49 minutes
Generated-but-failed cases	6 test file(s) did not compile or run

Engine warnings (4)

- ! Unexpected error while running fuzz test for generated file. File: TestCases_FablesStableHookLite_WeakRandomness
- ! Unexpected error while running fuzz test for generated file. File: TestCases_FablesStableHookLite_UnauthorizedStateModification
- ! Unexpected error while running fuzz test for generated file. File: TestCases_FablesStableHookLiteTyped_WeakRandomness
- ! Unexpected error while running fuzz test for generated file. File: TestCases_FablesStableHookLiteTyped_UnauthorizedStateModification

OLYMPIX
Proactive Web3 smart contract security

© 2026 Olympix Inc. All rights reserved. · Prepared for FablesFi · Session `79939a21-05b3-4a3f-9d25-c3ffde466cda` · November 11, 2026 UTC · Every figure in this report is reproducible from the artifacts shipped alongside it.